

01_structure_sources

1. dbt Project Structure

Helios is an ELT system whose entire "T" is one dbt project. The repository layout is not cosmetic: it encodes the layered DAG (`src_ga4` → `stg_ga4_*` → `int_ga4_*` → `fct_*/dim_*` → `semantic_layer.yaml`), the materialization strategy, and the governance boundary that keeps the agents on rails. Every file below maps to a pinned artifact in the model catalog; nothing is invented at run time.

Project tree

```
helios/
├── dbt_project.yml          # project config: paths, folder-level configs, vars
├── packages.yml            # dbt-utils, dbt_expectations, dbt_project_evaluator, codegen
├── profiles.yml            # dev→helios_dev (oauth), prod→helios_prod (WIF/SA)
├── selectors.yml           # named selectors (slim CI, layer builds)
├── seeds/
│   └── channel_group_mapping.csv # canonical source→channel mapping (feeds channel_group_case)
├── macros/
│   ├── get_event_param.sql  # get_event_param(key, type) typed extraction
│   ├── sessionize.sql       # sessionize(): session_key = TO_HEX(MD5(...))
│   ├── channel_group.sql    # channel_group_case(): SINGLE SOURCE OF TRUTH, 10 groups
│   └── test_revenue_reconciles.sql # custom generic test: mart total == source total
├── models/
│   ├── staging/
│   │   ├── _ga4_sources.yml # src_ga4 declaration (sources + freshness)
│   │   ├── _ga4_models.yml  # staging schema.yml: docs, tests, columns
│   │   ├── _ga4_docs.md     # doc blocks reused across layers
│   │   ├── stg_ga4_events.sql # view: 1:1 typed/renamed events
│   │   └── stg_ga4_event_params.sql # view: unnested event_params long table
│   ├── intermediate/
│   │   ├── _int_ga4_models.yml # intermediate schema.yml (tests, NOT exposed to BI)
│   │   ├── int_ga4_sessionized.sql # KEYSTONE: session grain, channel/landing/source
│   │   └── int_ga4_funnel_steps.sql # KEYSTONE: reached_* monotonic flags + session_revenue
│   └── marts/
│       ├── core/
│       │   ├── _core_models.yml # contracts, tests, exposures-facing docs
│       │   ├── fct_sessions.sql  # incremental: session PK session_key
│       │   ├── fct_funnel.sql    # incremental: primary session grain for semantic
│       │   ├── fct_daily_funnel.sql # incremental: additive day x dim step counts
│       │   ├── dim_users.sql     # table: user_pseudo_id (PK user_key)
│       │   ├── dim_items.sql     # table: item_id (PK item_key)
│       │   └── dim_channels.sql  # table: 10 channel groups (PK channel_key)
```

```

├── dim_date.sql          # table: conformed date spine
├── finance/
│   ├── _finance__models.yml
│   ├── fct_orders.sql    # table: transaction_id (PK order_key)
│   └── fct_order_items.sql # table: order line (PK order_item_key)
├── growth/
│   ├── _growth__models.yml
│   ├── fct_funnel_by_dim.sql # table: funnel rollup by canonical dimension
│   └── fct_cohorts.sql      # table: weekly acquisition cohorts
├── semantic/
│   └── semantic_layer.yml  # governed registry: 47 metrics / 19 dims (MetricFlow)
├── snapshots/
│   └── snap_dim_items.sql  # SCD2 on item price/category
├── tests/
│   ├── assert_session_conversion_rate_bounds.sql # singular: 0 ≤ rate ≤ 1
│   └── assert_funnel_monotonicity.sql            # singular: reached_* monotone
├── analyses/
│   └── adhoc_mix_shift_explore.sql # compiled-only exploration, never materialized
├── exposures/
│   └── _exposures.yml          # 7 agents + the Decision Brief
├── .github/
│   └── workflows/
│       ├── ci.yml             # slim CI: state:modified+ --defer + dbt test
│       └── prod_build.yml     # merge-to-main prod refresh + source freshness gate

```

The split of YAML config files (`_ga4__sources.yml` , `_ga4__models.yml` , `_core__models.yml` , ...) follows the dbt convention of one config file per folder, prefixed with `_` so it sorts to the top. Exposures and sources are deliberately separated from model `schema.yml` files to keep each file single-purpose.

dbt_project.yml

Folder-level `+config` blocks set materialization, partitioning, clustering, grouping/access, tags, and `persist_docs` once per layer, so individual model files carry only the overrides they actually need. This is where the BigQuery cost controls and the access boundary between layers are declared.

```

name: helios
version: '1.0.0'
config-version: 2
require-dbt-version: '≥1.7.0'

profile: helios

model-paths: ['models']
seed-paths: ['seeds']
macro-paths: ['macros']
snapshot-paths: ['snapshots']
test-paths: ['tests']

```

```

analysis-paths: ['analyses']

clean-targets: ['target', 'dbt_packages']

vars:
  # Build window - matches the static public sample; prod overrides via --vars.
  ga4_start_date: '2020-11-01'
  ga4_end_date: '2021-01-31'
  # Late-arrival lookback for incremental insert_overwrite.
  incremental_lookback_days: 3
  # Engaged-session threshold (msec); keeps the rule in one place.
  engagement_time_threshold_msec: 10000
  # dbt_project_evaluator scoping
  'dbt_project_evaluator:exclude_packages': true

# Surrogate-key salt + dispatch so dbt_utils macros resolve on BigQuery first.
dispatch:
  - macro_namespace: dbt_utils
    search_order: ['helios', 'dbt_utils']

models:
  +persist_docs:
    relation: true
    columns: true

  helios:
    staging:
      +materialized: view
      +group: staging
      +access: private # nothing outside staging may ref a stg_model
      +tags: ['staging', 'ga4']
      +schema: staging

    intermediate:
      +materialized: ephemeral # inlined; never exposed to BI
      +group: intermediate
      +access: private
      +tags: ['intermediate', 'ga4']

  marts:
    +schema: marts
    core:
      +materialized: incremental
      +incremental_strategy: insert_overwrite
      +partition_by:
        field: event_date
        data_type: date
        granularity: day
      +cluster_by: ['device_category', 'channel_group']
      +require_partition_filter: true
      +on_schema_change: append_new_columns
      +group: marts
      +access: public # the semantic layer + exposures consume these

```

```

+tags: ['marts', 'core']
# Dims are small + non-partitioned: override to table.
dim_users:    { +materialized: table, +require_partition_filter: false }
dim_items:    { +materialized: table, +require_partition_filter: false }
dim_channels: { +materialized: table, +require_partition_filter: false }
dim_date:     { +materialized: table, +require_partition_filter: false }

finance:
  +materialized: table
  +group: marts
  +access: public
  +tags: ['marts', 'finance']

growth:
  +materialized: table
  +group: marts
  +access: public
  +tags: ['marts', 'growth']

semantic:
  +materialized: view
  +group: marts
  +access: public
  +tags: ['semantic']

seeds:
  helios:
    channel_group_mapping:
      +column_types:
        source: string
        medium: string
        channel_group: string

snapshots:
  helios:
    +target_schema: snapshots
    +tags: ['snapshot']

```

Two things are load-bearing here. First, `+access: private` on staging and intermediate is enforced by dbt's **groups/access** feature: any attempt to `ref('stg_ga4__events')` from a mart in a different group fails compilation, structurally preventing the "no layer reaches across or upward" rule from being violated. Marts are `public` so the semantic layer and exposures can consume them. Second, `require_partition_filter: true` on the partitioned core facts forces every downstream query (including ad-hoc ones the warehouse-mcp issues) to supply an `event_date` predicate, which is the primary defense against a full-table scan blowing the byte budget. The four dims override `require_partition_filter` back to `false` because they are unpartitioned tables.

packages.yml

```

packages:
- package: dbt-labs/dbt_utils
  version: ["≥1.1.0", "<2.0.0"]
- package: calogica/dbt_expectations
  version: ["≥0.10.0", "<0.11.0"]
- package: dbt-labs/dbt_project_evaluator
  version: ["≥0.14.0", "<0.15.0"]
- package: dbt-labs/codegen
  version: ["≥0.12.0", "<0.13.0"]

```

- **dbt_utils** supplies `generate_surrogate_key` (used for `session_key` callers that need a deterministic composite key), `accepted_range`, `equal_rowcount`, and `unique_combination_of_columns` (the test that proves the grain of `fct_daily_funnel` is genuinely `day x channel_group x device_category x country x is_new_user`).
- **dbt_expectations** adds distributional and statistical tests (`expect_column_values_to_be_between`, `expect_column_values_to_not_be_null`, `expect_row_values_to_have_recent_data`) that data-layer tests rely on for rate bounds and freshness.
- **dbt_project_evaluator** is run in CI to enforce structural conventions automatically: it flags a mart that refs a source directly, a model with no description, a staging model not named `stg_*`, or a fan-out join — catching layering violations the access groups can't.
- **codegen** generates boilerplate (`generate_source`, `generate_model_yaml`) so new staging columns are scaffolded, not hand-typed.

After cloning, `dbt deps` installs these into `dbt_packages/`. CI runs `dbt deps` before every build.

profiles.yml

Two targets, two BigQuery datasets, two auth methods. Developers authenticate interactively with OAuth (no keys on laptops); production authenticates via **Workload Identity Federation** from GitHub Actions, so there are no long-lived service-account JSON keys anywhere in the system.

```

helios:
  target: dev
  outputs:
    dev:
      type: bigquery
      method: oauth                # gcloud ADC; no key files on developer machines
      project: helios-analytics
      dataset: helios_dev          # personal/shared dev marts
      location: US
      threads: 8
      priority: interactive

```

```

job_execution_timeout_seconds: 600
job_retries: 1
maximum_bytes_billed: 5368709120 # 5 GiB hard cap per query (the run byte budget)

prod:
  type: bigquery
  method: oauth # WIF-issued ADC token in CI; no JSON key
  project: helios-analytics
  dataset: helios_prod
  location: US
  threads: 16
  priority: batch # cheaper, non-interactive for scheduled refresh
  job_execution_timeout_seconds: 1800
  job_retries: 2
  maximum_bytes_billed: 21474836480 # 20 GiB ceiling for the full prod refresh

```

`location: US` is mandatory and must match `bigquery-public-data` (the GA4 sample lives in the US multi-region); a mismatch causes "dataset not found in location" errors. `priority: interactive` in dev gives developers fast feedback; `priority: batch` in prod queues jobs against idle slots to lower cost. `maximum_bytes_billed` is set per target — 5 GiB in dev maps directly to the project's per-run byte budget so a careless dev query fails fast rather than silently costing money.

In CI, `method: oauth` consumes a short-lived access token minted by the `google-github-actions/auth` step using WIF; dbt reads it from Application Default Credentials. No `keyfile` path appears anywhere.

Conventions & naming

These are the rules that make the model catalog self-consistent. They are enforced by a mix of `dbt_project.yml` config, `dbt_project_evaluator`, and code review.

- **Layer prefixes.** `stg_<source>__<entity>` for staging, `int_<source>__<entity>` for intermediate, `fct_*` / `dim_*` for marts. The source group is `src_ga4`. Everything is `snake_case` — files, models, columns, tests.
- **One model per file.** A `.sql` file produces exactly one relation, named identically to the file. No model defines two `SELECT` s; reusable logic lives in macros or an intermediate model.
- **No cross-layer or upward refs.** A model may only `ref()` models in the layer immediately upstream (or the same layer for intermediate fan-in). Marts never `ref()` a source — they go through staging. This is enforced structurally by `+access: private` on staging/intermediate and audited by `dbt_project_evaluator`'s `fct_rejoining_of_upstream_concepts` / direct-source-dependency checks.

- **session_key.** The one canonical expression, emitted by the `sessionize()` macro and used nowhere else by hand:

```
session_key = TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))
```

Session counts are always `COUNT(DISTINCT session_key)` — never `FARM_FINGERPRINT`, never `COUNT(*)`.

- *reached_funnel flags are MAX-DOWNSTREAM (monotonic).* `reached_add_to_cart` is true if the session fired `add_to_cart` or any later stage. This guarantees `sessions ≥ reached_view_item ≥ reached_add_to_cart ≥ ... ≥ reached_purchase`, so every step rate is ≤ 1 by construction. The retired `did_*` naming is forbidden. `assert_funnel_monotonicity` enforces this at build time.
- **engaged_session** = `session_engaged = '1'` OR `engagement_time_msec ≥ var('engagement_time_threshold_msec')` (10000) — the threshold lives in `vars`, defined once.
- **Channel grouping lives in exactly one macro.** `channel_group_case()` (in `macros/channel_group.sql`) is the single source of truth for the 10 groups — Direct, Organic Search, Paid Search, Display, Paid Social, Organic Social, Email, Affiliates, Referral, Other. It is `has_gclid`-aware and backed by `seeds/channel_group_mapping.csv`. No `CASE` statement that buckets channels may exist anywhere else; an 11th group is a hard error.
- **Money & rates.** Only `*_in_usd` columns are aggregated. Rates are computed as `SUM(numerator)/SUM(denominator)` after grouping — never an average of per-segment ratios (this is the Simpson's-paradox defense). The `traffic_source` gotcha: event-level `traffic_source.*` is user first-touch, not session source — prefer session-scoped `event_params` source/medium and fall back to `traffic_source` only when null.
- **Marts are wide.** Each fact carries its descriptive dimensions denormalized (device, channel, geo, date) so the semantic layer slices without runtime joins.

Environments

Two physical BigQuery datasets isolate work, selected by dbt target in `profiles.yml`:

Concern	dev	prod
dataset	<code>helios_dev</code>	<code>helios_prod</code>
auth	OAuth / gcloud ADC	WIF-minted ADC (no keys)
priority	interactive	batch

Concern	dev	prod
<code>maximum_bytes_billed</code>	5 GiB	20 GiB
<code>build window (vars)</code>	full static sample	overridden to live window

Alongside the two dbt targets, the platform reads/writes the `marts`, `semantic`, `helios_memory`, and `helios_eval` schemas; the semantic layer (`semantic_layer.yaml`) is identical across environments so the agents behave consistently dev-to-prod.

Defer to prod. Slim CI builds only changed models and their children with `--defer --state ./prod-manifest`, resolving unbuilt upstream `ref()` s against the production relations. A developer can build a single mart in `helios_dev` while its (unchanged) upstream staging models resolve to `helios_prod`, so no one has to materialize the entire DAG to test one model.

Blue-green / Write-Audit-Publish. At the session/day grain Helios operates, partition-atomic `insert_overwrite` already gives a clean swap — a failed incremental run leaves prior partitions untouched and the marts queryable. Full blue-green or a WAP staging dataset is therefore not warranted here; we note it as the upgrade path if Helios ever serves a live external dashboard where a partial mart must never be visible. For now, `insert_overwrite` idempotency plus the source-freshness gate (Section 2) is the correctness boundary.

2. Source Models

A dbt project must never `ref()` or hard-code a raw warehouse table. Instead it **declares sources** in a `sources.yml` file, then references them with `source('src_ga4', 'events')`. This gives three things for free: a single place to repoint the warehouse (public sample today, your own live GA4 export tomorrow), source-level freshness SLAs that can gate the build, and source documentation that flows into the generated docs site and lineage graph. For Helios, the source is the date-sharded GA4 export, and the source declaration is the contract that fails loudly the day Google changes the export schema.

`sources.yml` for `src_ga4`

```
# models/staging/_ga4__sources.yml
version: 2

sources:
  - name: src_ga4
    description: >
      GA4 BigQuery Export (Google Analytics 4 e-commerce events). In production this
```



```

is the project's own live daily export dataset; on the public sample it is the
static, obfuscated Google Merchandise Store export covering 2020-11-01..2021-01-31.
database: bigquery-public-data           # prod: your own GCP project
schema: ga4-obfuscated-sample-ecommerce  # prod: analytics_<property_id>
loader: GA4 BigQuery Export
loaded_at_field: parse_date('%Y%m%d', event_date)

# PRODUCTION freshness SLA - informational only on the static sample (see note).
freshness:
  warn_after: { count: 36, period: hour }
  error_after: { count: 48, period: hour }

tables:
  - name: events
    description: >
      Date-sharded GA4 event stream. One physical table per day,
      events_YYYYMMDD; the wildcard identifier unions them. Grain = one row
      per event per user per session. Carries nested event_params, items,
      device, geo, traffic_source, and ecommerce STRUCT/ARRAY fields.
    identifier: 'events_*'                # wildcard over the date shards
    columns:
      - name: event_date
        description: 'Shard date as YYYYMMDD string; also exposed via _TABLE_SUFFIX.'
        tests:
          - not_null
      - name: event_timestamp
        description: 'Event time, microseconds since UNIX epoch (INT64).'
        tests:
          - not_null
      - name: event_name
        description: 'GA4 event name (e.g. session_start, view_item, purchase).'
        tests:
          - not_null
      - name: user_pseudo_id
        description: 'Cookie/device-grain pseudonymous user id; basis of session_key.'
        tests:
          - not_null
      - name: ecommerce.transaction_id
        description: 'Purchase transaction id; null for non-purchase events.'

# Loud-failure contract: alert if the raw event volume collapses.
tests:
  - dbt_utils.recency:
      datepart: day
      field: parse_date('%Y%m%d', event_date)
      interval: 2
      config:
        severity: warn          # warn on the static sample; error in prod overlay

```

The source declaration carries the schema contract (the columns the staging layer depends on are named and `not_null` -tested) and the freshness SLA. Staging models reference it as `{{`

`source('src_ga4', 'events') }}`; no staging model contains a literal `bigquery-public-data...` reference.

Declaring sources, never raw refs

The rule is absolute: **staging is the only layer allowed to touch the source, and it does so only through `source()`**. Concretely, `stg_ga4__events` begins:

```
-- models/staging/stg_ga4__events.sql
with source as (
  select *
  from {{ source('src_ga4', 'events') }}
  where _table_suffix between
    replace('{{ var("ga4_start_date") }}', '-', '')
    and replace('{{ var("ga4_end_date") }}', '-', '')
)
```

This single indirection is what lets the same dbt code run against the public sample in dev and a real GA4 export in prod by changing only `database / schema` in `sources.yml`. It also makes the lineage graph correct: `dbt docs` shows `src_ga4.events` → `stg_ga4__events` → ... and `dbt_project_evaluator` can flag any model that skips the source.

Date-sharded wildcard + `_TABLE_SUFFIX` pruning

GA4 exports one physical table per day, `events_YYYYMMDD`. The `identifier: 'events_*` wildcard tells BigQuery to union them, and the pseudo-column `_TABLE_SUFFIX` (the part matched by `*`, e.g. `20210114`) is the **partition-pruning lever**. Filtering on `_TABLE_SUFFIX` makes BigQuery scan only the matching shards — the single biggest cost control in the project, because an unfiltered `events_*` scans the entire history.

```
-- prune shards: scan only the build window (dev) or the incremental lookback (prod)
where _table_suffix between '20201101' and '20210131'
```

On incremental prod runs the predicate narrows further to the lookback window so only newly landed shards are scanned:

```
{% if is_incremental() %}
  -- only reprocess the last N days of shards (late-arrival window)
  where _table_suffix ≥ format_date(
    '%Y%m%d',
    date_sub(_dbt_max_partition, interval {{ var('incremental_lookback_days') }} day)
```

```
)  
{% endif %}
```

Combined with `require_partition_filter` on the downstream facts and `maximum_bytes_billed`, this keeps a full pipeline run inside the 5 GiB-per-query budget. We never `SELECT *` into a mart; staging selects an explicit column list so unused nested fields aren't scanned.

Source documentation

Descriptions on the source, table, and columns (above) render into the dbt docs site and the lineage DAG, and `persist_docs` pushes the table/column descriptions into BigQuery's own metadata so they appear in the BigQuery console too. This is the human-readable contract: when a developer or agent asks "what is `user_pseudo_id`?", the answer lives next to the declaration, not in tribal knowledge. Doc blocks in `_ga4_docs.md` (e.g. a `session_key` definition) are referenced with `{{ doc('session_key') }}` so the explanation is written once and reused across every layer that surfaces the column.

Source freshness and how it gates the build

`loaded_at_field` is `parse_date('%Y%m%d', event_date)` — the freshest event date becomes the table's "load time." `dbt source freshness` compares the max `event_date` against `warn_after` (36h) / `error_after` (48h) and writes a `sources.json` result:

```
# Run freshness checks against the declared SLA  
dbt source freshness  
  
# Gate the production build on it: only build if the source is fresh.  
dbt source freshness && dbt build --target prod
```

In the scheduled prod pipeline, `dbt source freshness` runs first and its exit code gates everything downstream. If the latest GA4 shard is older than 48 hours the command exits non-zero, the build is aborted, and — critically — the agent pipeline does **not** run. A stale source must block diagnosis rather than let Monitor/Decompose silently reason over yesterday's (or last week's) data. The freshness result also feeds the slim-CL selector `source_status:fresher+`, which can restrict a run to only the models downstream of sources that have new data.

Production note (honesty about the static sample)

The pinned freshness thresholds describe the **production** strategy: in prod, `src_ga4` points at the project's own GA4 export dataset (`analytics_<property_id>`), which lands a new

`events_YYYYMMDD` shard daily, and a 36h/48h SLA correctly catches a stalled export.

On the **public sample**, `bigquery-public-data.ga4_obfuscated_sample_ecommerce` is static and historical — the newest shard is `events_20210131` and it never updates. Real freshness against "now" (2026) is therefore meaningless: every freshness check would scream `error`. So on the sample the freshness declaration is **informational only**, and the project degrades gracefully:

- The `dbt_utils.recency` source test is set to `severity: warn` (not `error`) so a static sample doesn't hard-fail every build.
- The prod pipeline's `dbt source freshness && dbt build` gate is enabled only on the `prod` target; the dev/sample build skips the gate (or runs `dbt source freshness` non-blocking, purely to exercise the code path).
- Documentation and tests still validate the *shape* of freshness — that `loaded_at_field` parses, that the SLA YAML is well-formed — so the moment Helios is repointed at a live export, the gate is already wired and correct with no code change beyond `sources.yml`'s `database / schema`.

This keeps the guide honest: the freshness machinery is real, production-grade, and fully implemented; on the frozen sample it is intentionally non-blocking because there is no "fresh" to measure.